

UFUG 2602

Advanced Sorting

Tengfei Chang, Jia Li, Xihu Zheng, Zijun Gong

IoT, DSA, INTR Thrusts
Hong Kong University of Science and Technology (Guangzhou)

Outline

- **The Master Theorem**
- Quick Sort
- Sorting Lower Bound

The Master Theorem: A Cake Cutting Story

Imagine you have a big cake to share with your friends, but it's too large to cut all at once. Here's how you can divide it:

- 1.Cut the Cake:** Each time, you cut the cake into α pieces of roughly equal size.
- 2.Process Each Piece:** You repeat the cutting for each piece until they're small enough to give to your friends.
- 3.Extra Work:** Each cutting round requires some extra tasks, like washing the knife or cleaning the table. This takes $g(n)$ time, where n is the current size of the cake.

Now, the question is: How long will the entire cake-cutting process take?



The Master Theorem

Let $f(n)$ be a function that returns a positive value for every integer $n > 0$. We know:

$$f(1) = O(1)$$
$$f(n) \leq \alpha \cdot f\left(\left\lceil \frac{n}{\beta} \right\rceil\right) + \underbrace{O(n^\gamma \cdot \log^\lambda n)}_{g(n)} \quad (\text{for } n \geq 2)$$

where $\alpha \geq 1, \beta > 1, \gamma \geq 0$, and $\lambda \geq 0$ are constants. Then:

- If $\log_\beta \alpha < \gamma$, then $f(n) = O(n^\gamma \log^\lambda n)$.
- If $\log_\beta \alpha = \gamma$, then $f(n) = O(n^\gamma \log^{\lambda+1} n)$.
- If $\log_\beta \alpha > \gamma$, then $f(n) = O(n^{\log_\beta \alpha})$.

α : The number of pieces the cake is divided into each time.

β : The factor by which the size of each piece is reduced compared to the original cake.

$g(n)$: The additional work required during each division.

The theorem can be proved by carefully applying the “expansion method” we saw earlier.

The details are tedious and omitted.

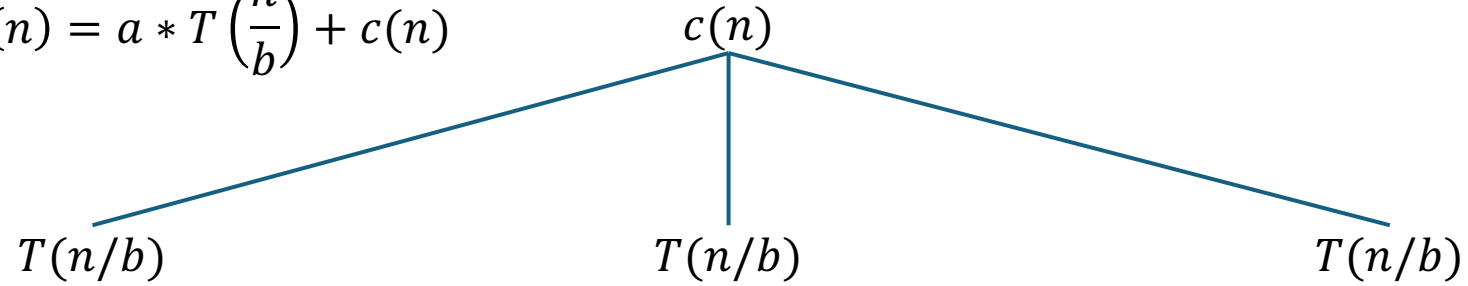
The Master Theorem

$$T(n) = b * T\left(\frac{n}{b}\right) + c(n)$$

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$

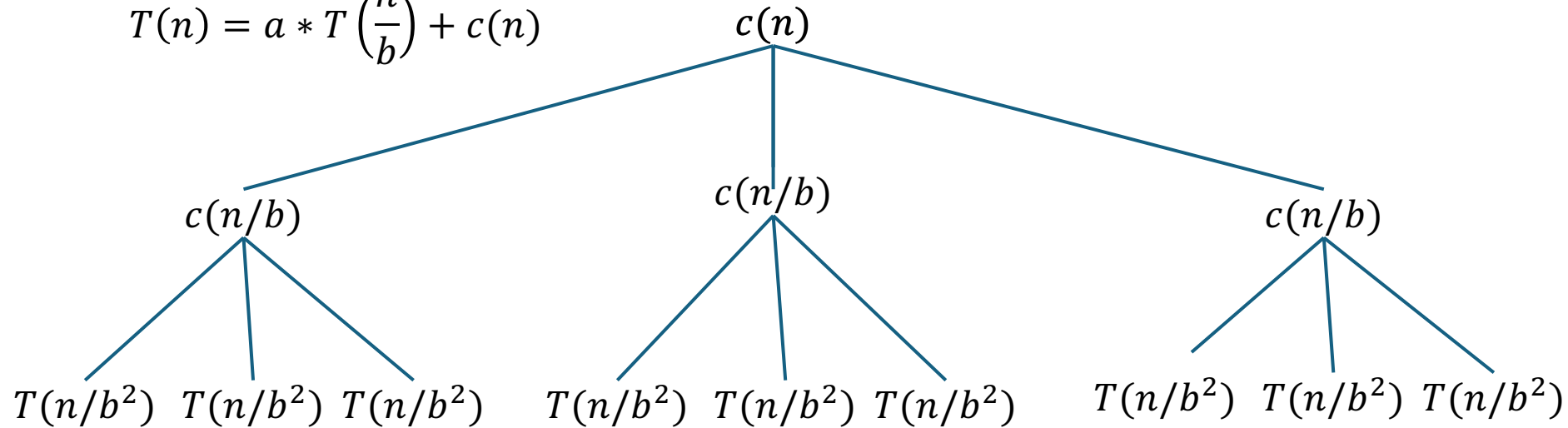
The Master Theorem

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$



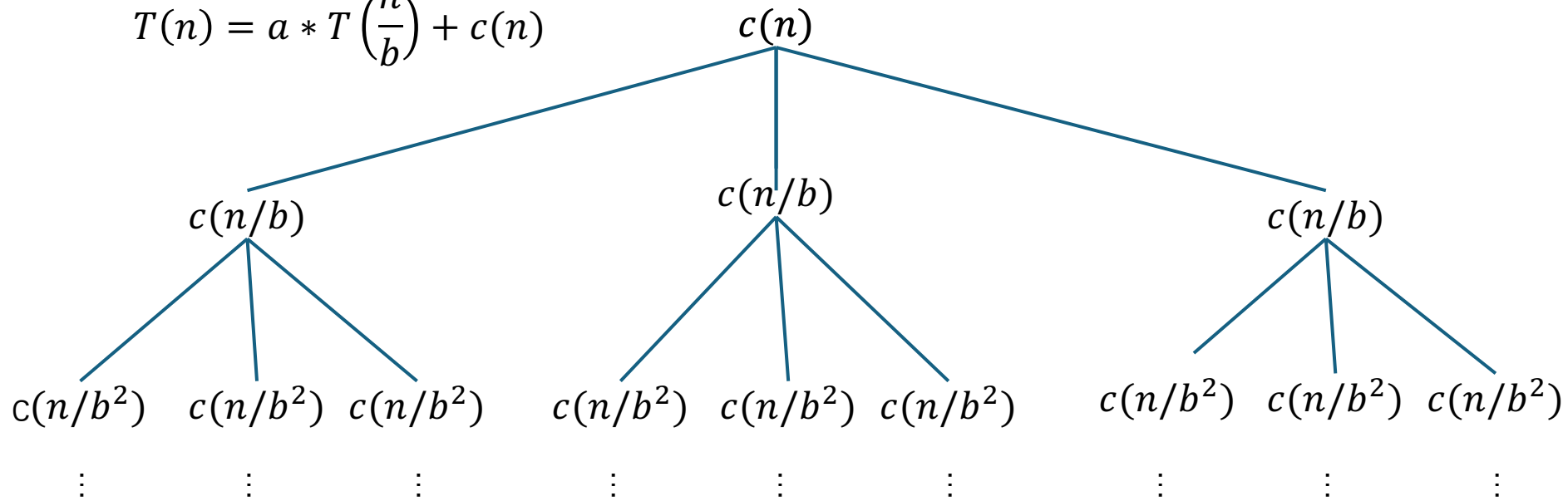
The Master Theorem

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$



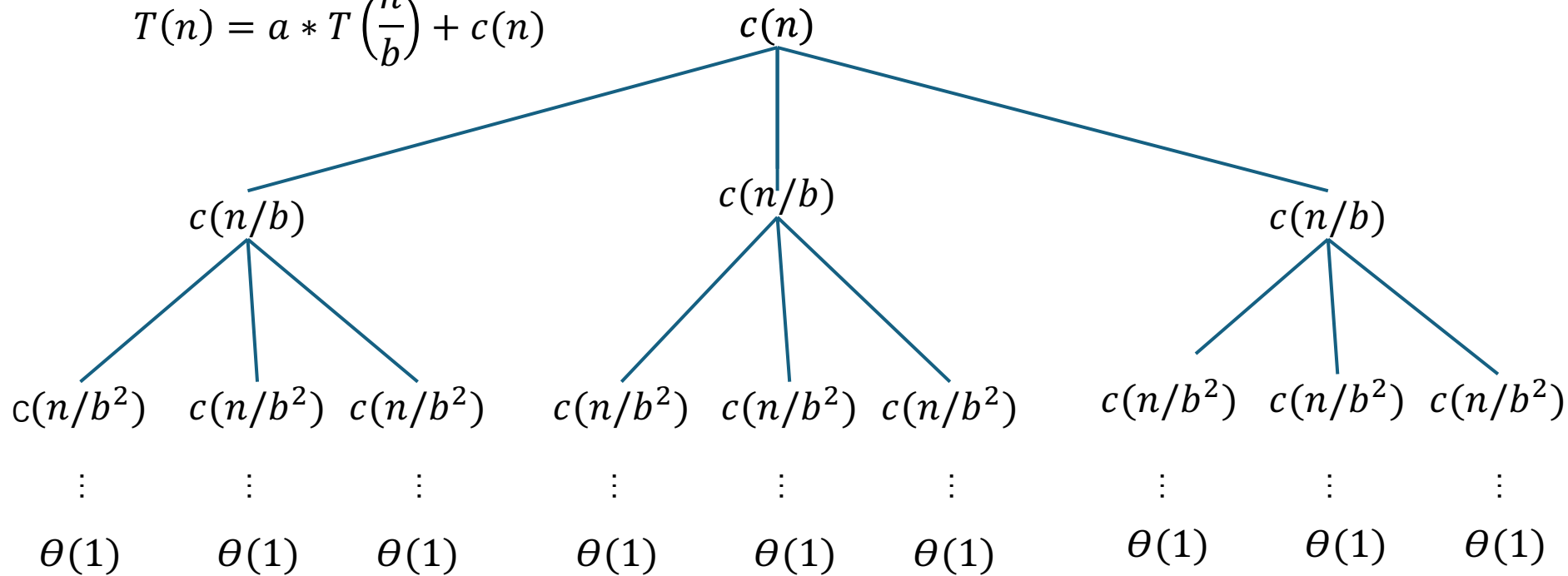
The Master Theorem

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$



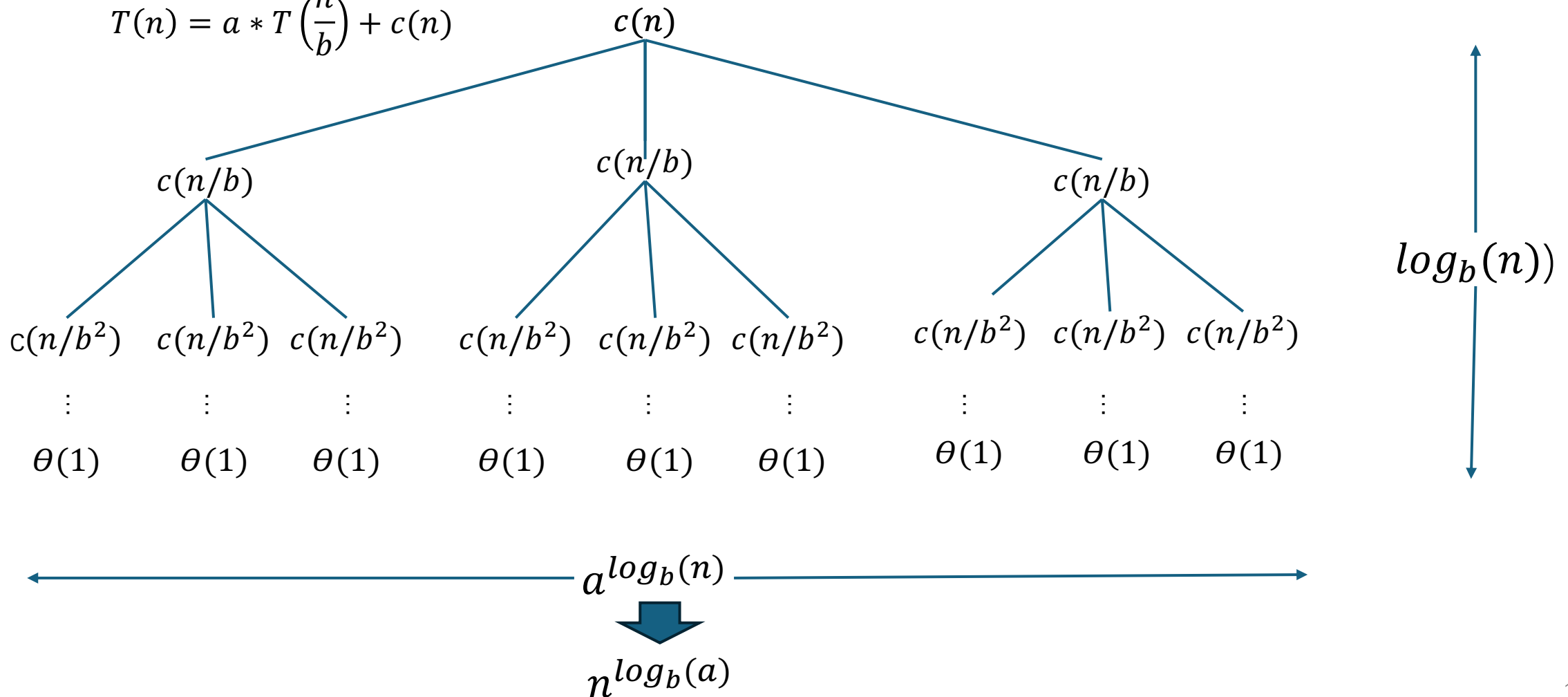
The Master Theorem

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$

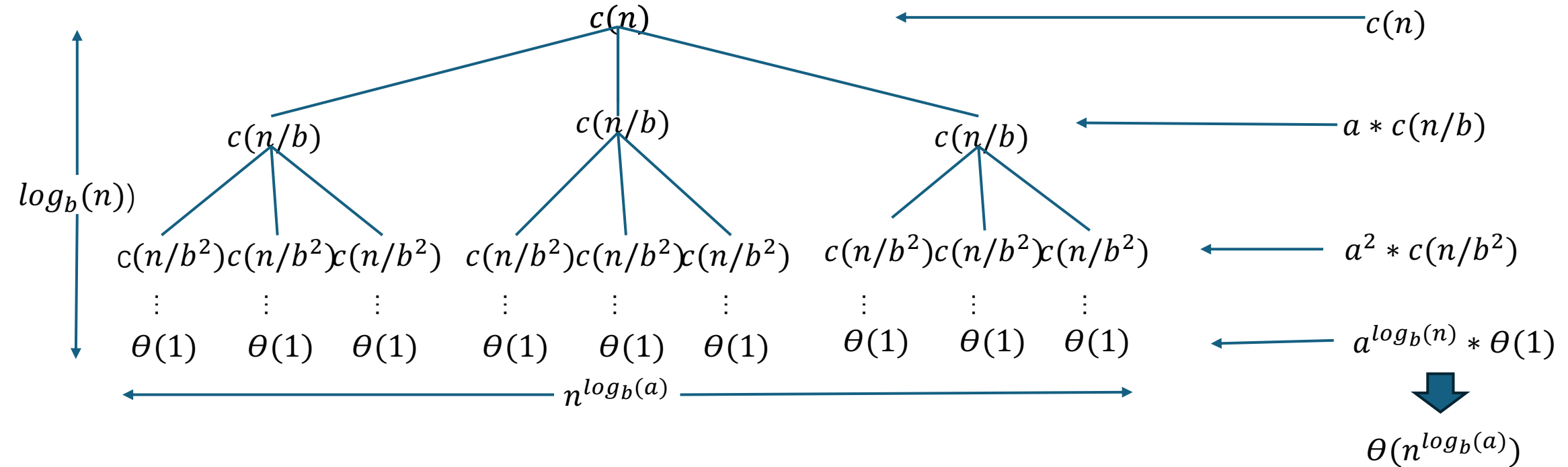


The Master Theorem

$$T(n) = a * T\left(\frac{n}{b}\right) + c(n)$$

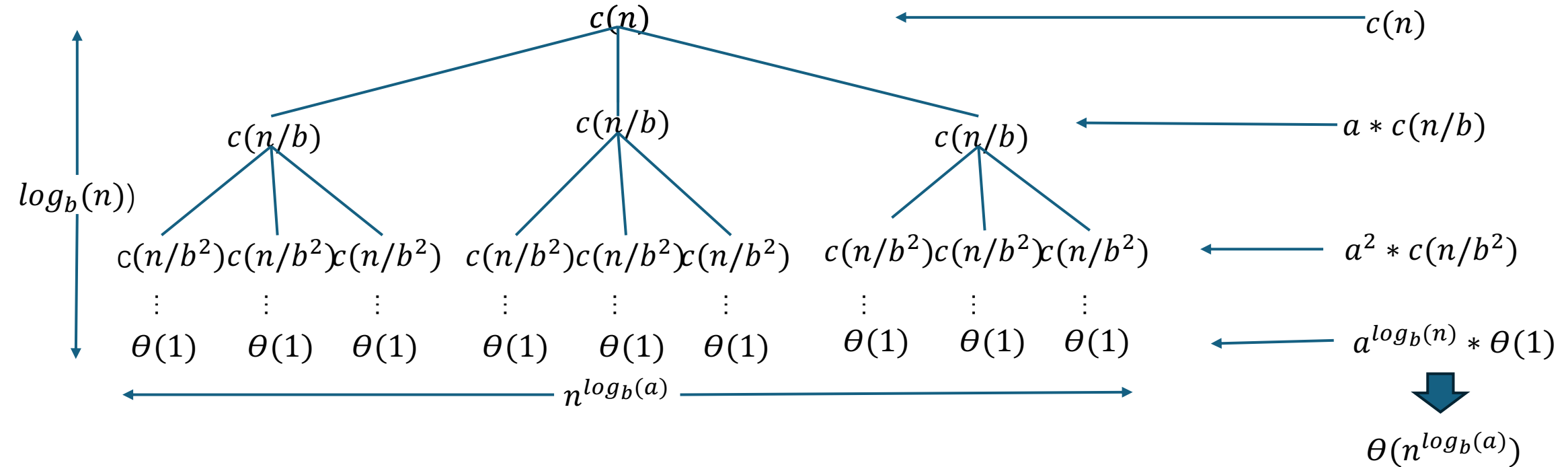


The Master Theorem



$$T(n) = c(n) + a * c\left(\frac{n}{b}\right) + a^2 * c\left(\frac{n}{b^2}\right) + \dots \theta(n^{\log_b(a)})$$

The Master Theorem



$$T(n) = \sum_{j=0}^{\log_b(n)-1} a^j * c(n/b^j) + \theta(n^{\log_b(a)})$$

The Master Theorem

$$T(n) = \theta(n^{\log_b(a)}) + \sum_{j=0}^{\log_b(n)-1} a^j * c(n/b^j)$$

1. If the problem gets easier -> Root is the hardest
2. If the hardness is evenly distributed -> All levels are relevant

$$T(n) = \theta(C(n))$$

$$T(n) = \text{height} * (\text{some level})$$

$$T(n) = \log_b(n) * \theta(n^{\log_b(a)}) = \theta(n^{\log_b(a)} \log_b(n))$$

3. If the problem gets harder -> The Leaf are the hardest

$$T(n) = \theta(n^{\log_b(a)})$$

The Master Theorem

Let $f(n)$ be a function that returns a positive value for every integer $n > 0$. We know:

$$\begin{aligned} f(1) &= O(1) \\ f(n) &\leq \alpha \cdot f\left(\left\lceil \frac{n}{\beta} \right\rceil\right) + \underbrace{O(n^\gamma \cdot \log^\lambda n)}_{g(n)} \quad (\text{for } n \geq 2) \end{aligned}$$

where $\alpha \geq 1$, $\beta > 1$, $\gamma \geq 0$, and $\lambda \geq 0$ are constants. Then:

- If $\log_\beta \alpha < \gamma$, then $f(n) = O(n^\gamma \log^\lambda n)$.
- If $\log_\beta \alpha = \gamma$, then $f(n) = O(n^\gamma \log^{\lambda+1} n)$.
- If $\log_\beta \alpha > \gamma$, then $f(n) = O(n^{\log_\beta \alpha})$.

α : The number of pieces the cake is divided into each time.

β : The factor by which the size of each piece is reduced compared to the original cake.

$g(n)$: The additional work required during each division.

The theorem can be proved by carefully applying the “expansion method” we saw earlier. The details are tedious and omitted.

The Master Theorem

Example 1:

Consider the recurrence of binary search:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq f(\lceil n/2 \rceil) + c_2 \quad (\text{for } n \geq 2) \end{aligned}$$

Hence, $\alpha = 1$, $\beta = 2$, $\gamma = 0$, and $\lambda = 0$. Since $\log_\beta \alpha = \gamma$, we know that $f(n) = O(n^0 \cdot \log^{0+1} n) = O(\log n)$.

The Master Theorem

Example 2:

Consider the recurrence of merge sort:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq 2 \cdot f(\lceil n/2 \rceil) + c_2 n \quad (\text{for } n \geq 2) \end{aligned}$$

Hence, $\alpha = 2, \beta = 2, \gamma = 1$, and $\lambda = 0$. Since $\log_\beta \alpha = \gamma$, we know that $f(n) = O(n^\gamma \cdot \log^{0+1} n) = O(n \log n)$.

The Master Theorem

Example 3:

Consider the recurrence:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq 2 \cdot f(\lceil n/4 \rceil) + c_2 \sqrt{n} \cdot \log_2 n \quad (\text{for } n \geq 2) \end{aligned}$$

Hence, $\alpha = 2$, $\beta = 4$, $\gamma = 1/2$, and $\lambda = 0$. Since $\log_\beta \alpha = \gamma$, we know that

$$f(n) = O(n^\gamma \cdot \log^{\lambda+1} n) = O(\sqrt{n} \log^2 n).$$

The Master Theorem

Example 4:

Consider the recurrence:

$$\begin{aligned} f(1) &\leq c_1 \\ f(n) &\leq 2 \cdot f(\lceil n/2 \rceil) + c_2 \sqrt{n} \quad (\text{for } n \geq 2) \end{aligned}$$

Hence, $\alpha = 2$, $\beta = 2$, $\gamma = 1/2$, and $\lambda = 0$. Since $\log_\beta \alpha > \gamma$, we know that $f(n) = O(n^{\log_\beta \alpha}) = O(n)$.

The Master Theorem

Example 5:

Consider the recurrence:

$$f(1) \leq c_1$$

$$f(n) \leq 13 \cdot f(\lceil n/7 \rceil) + c_2 n^2 \quad (\text{for } n \geq 2)$$

Hence, $\alpha = 13, \beta = 7, \gamma = 2$, and $\lambda = 0$. Since $\log_\beta \alpha < \gamma$, we know that $f(n) = O(n^\gamma \log^\lambda n) = O(n^2)$.

Q1. Apply Master Theorem

- For the recurrence:

$$T(n) = 4T(n/2) + n$$

- **Question:**
Use the Master Theorem to determine the time complexity.

Q2. Which Case of Master Theorem?

- For the recurrence:

$$T(n) = 2T(n/2) + n \log n$$

- **Question:**
- Which case does it belong to?
- What is the final complexity?

Q3. True or False

- For the recurrence:

$$T(n) = 8T(n/2) + n^2$$

- **Statement:**
“The time complexity is $\Theta(n^2)$ ”.

Q4. Turn Story into Recurrence

- An algorithm:
 - splits the problem into **3 equal subproblems**
 - each subproblem has size $n/4$
 - spends n extra work to combine
- **Question:**
 - Write the recurrence
 - Solve it with Master Theorem

Q5. Master Theorem Questions

- For the recurrence:

$$T(n) = 5T(n/2) + n^2$$

$$T(n) = 2T(n/4) + \sqrt{n}$$

- **Question:**
 - Which case does it belong to?
 - What is the final complexity?

Outline

- The Master Theorem
- **Quick Sort**
- Sorting Lower Bound

Quick Sort: Organizing the bookshelf



1. Choose a Pivot: Pick a random book as the pivot (e.g., a medium-height book).

2. Partition: Divide the books into two groups:

1. **Left group:** Books shorter than the pivot.

2. **Right group:** Books taller than the pivot.

The pivot stays in place as its final position is now fixed.

3. Recursively Repeat:

1. For the left group, pick a new pivot and divide it further.

2. For the right group, pick a new pivot and divide it further.

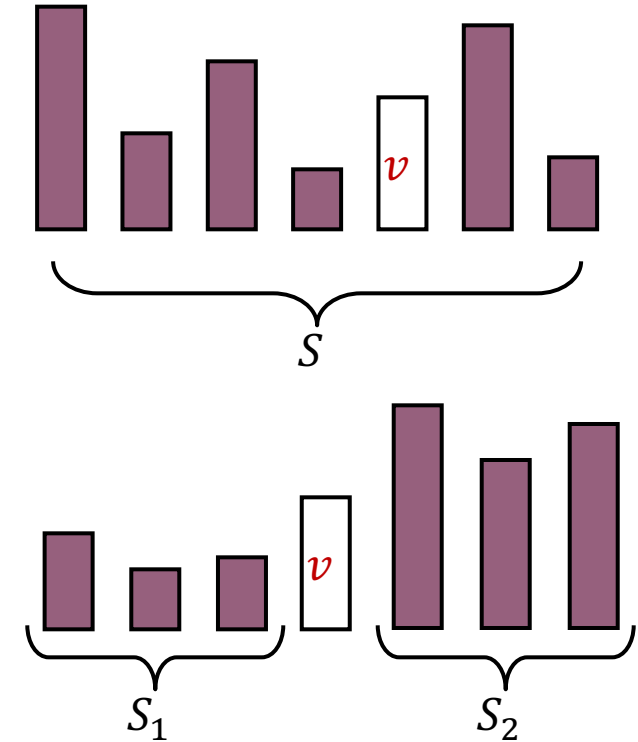
3. Repeat until each group has only one or no book.

4. Combine: Arrange the books in order: left group + pivot + right group.



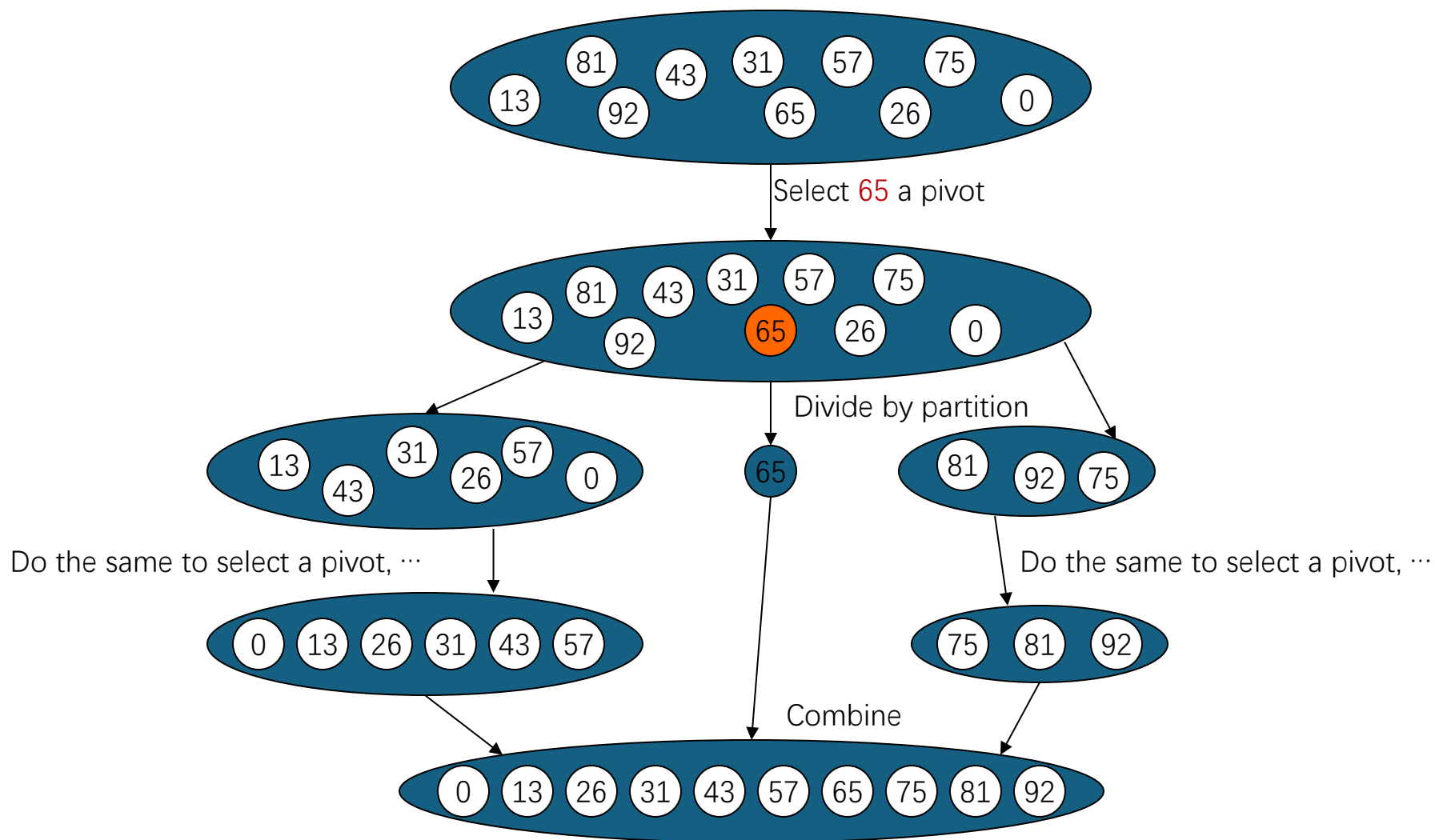
Quick Sort: Divide and Conquer

- Divide & Conquer.
 - Assume that all numbers are distinctive numbers.
- Divide step:
 - Divide a set S into two subsets S_1 and S_2 .
 - Pick **any number** v as a **pivot** in S .
 - **Partition** $S \setminus \{v\}$ into two sets:
 - $S_1 = \{x \in S \setminus \{v\} \mid x < v\}$ and $S_2 = \{x \in S \setminus \{v\} \mid x > v\}$.
- Conquer step: sort S_1 and S_2 recursively.



- Combine step: sort S by the sorted S_1 followed by v , and followed by the sorted S_2 .
 - Don't need to do much in this step.

A Quick Sort Example



Q6: Quick Sort Trace

Run **one partition only** on:

[9, 4, 7, 3, 8, 2]

using the **first element as pivot**.

Quick Sort: Divide and Conquer

- The pseudo code.

QuickSort(a, left, right):

if left < right:

select a pivot v in $a[\text{left}..\text{right}]$

move $a[i] < v$ to left ($a[\text{left}..p-1]$)

move $a[i] \geq v$ to right ($a[p+1..\text{right}]$)

$a[p] = v$ /* $a[p]$ keeps the pivot value v */

quickSort(a, left, p-1)

quickSort(a, p+1, right)

Partition



- Question 1: How to do the partition efficiently?
- Question 2: How to select the pivot?

Quick Sort: Partition

- Suppose we use an additional array $b[]$ to do the partition for $a[]$ of size n .
- Let the pivot value be p .
- Let $i = 0$ and $j = n-1$.
- Scan $a[k]$ from $k=0$ to $n-1$.
 - If $a[k] < p$, then place it at $b[i++]$ from the left.
 - If $a[k] > p$, place it at $b[j--]$ from the right.
- Finally, place p at the remaining slot.

Quick Sort: Partition

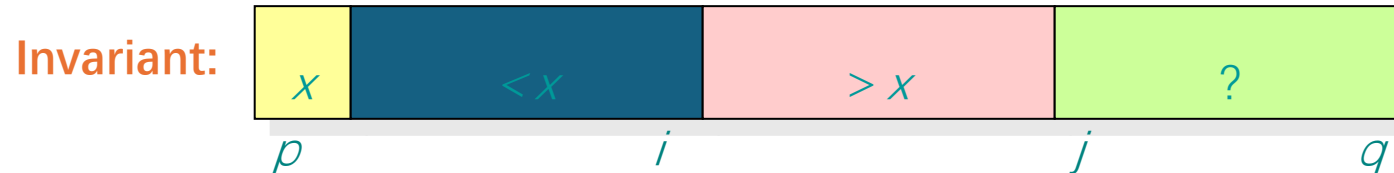
- Suppose we use an **additional array** $b[]$ to do the partition for $a[]$ of size n .
- Let the pivot value be p .
- Let $i = 0$ and $j = n-1$.
- Scan $a[k]$ from $k=0$ to $n-1$:
 - If $a[k] < p$, then place it at $b[i++]$ from the left;
 - Otherwise, place it at $b[j--]$ from the right.
- Finally, place p at the remaining slot.
- The partition time is $O(n)$
- **What is the problem?** Take additional space of $b[]$.

A More Space-efficient Partition

Without loss of generality, assume the first element is the pivot.

```
Partition(A,p,q): #A[p..q]
   $x \leftarrow A[p]$       #pivot =  $A[p]$ 
   $i \leftarrow p$ 
  for  $j \leftarrow p + 1$  to  $q$ 
    do if  $A[j] < x$ 
      then  $i \leftarrow i + 1$ 
      exchange  $A[i] \leftrightarrow A[j]$ 
  exchange  $A[p] \leftrightarrow A[i]$ 
  return  $i$ 
```

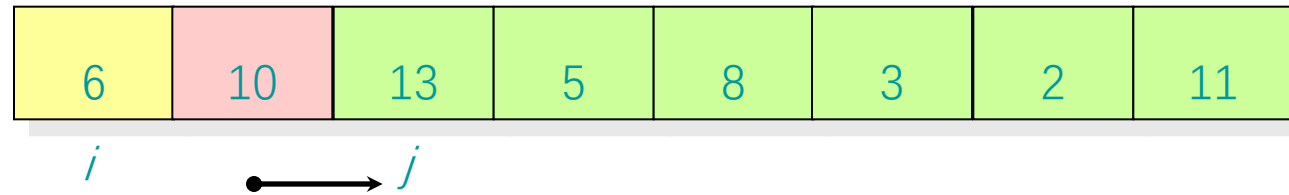
- Running time = $O(n)$ for n elements.
- The extra space is constant



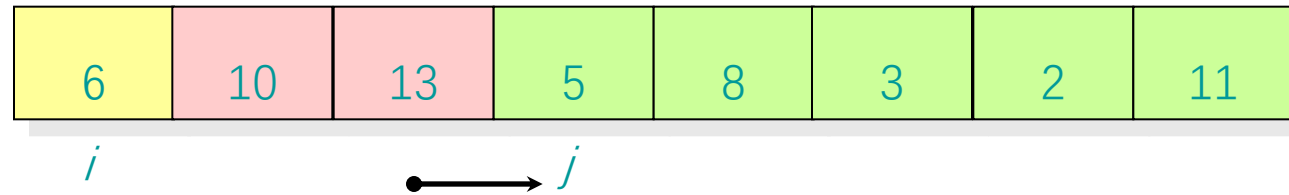
Example of partitioning

6	10	13	5	8	3	2	11
<i>i</i>	<i>j</i>						

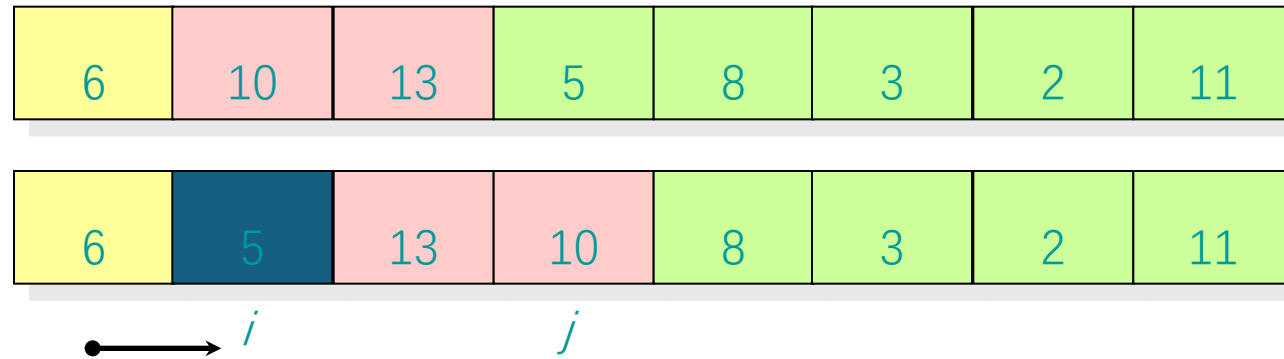
Example of partitioning



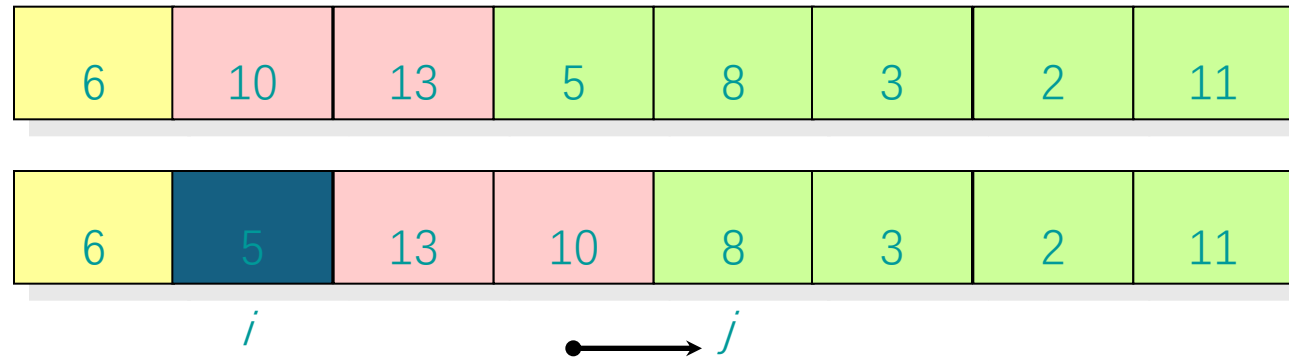
Example of partitioning



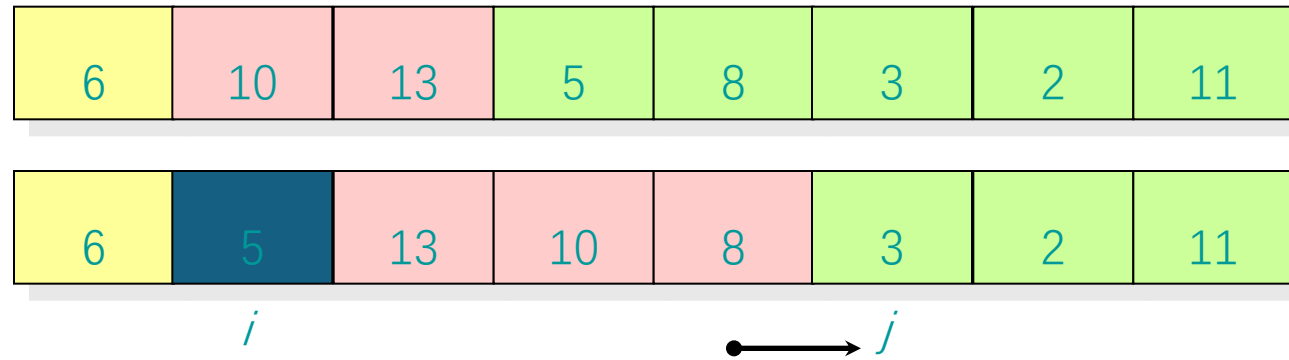
Example of partitioning



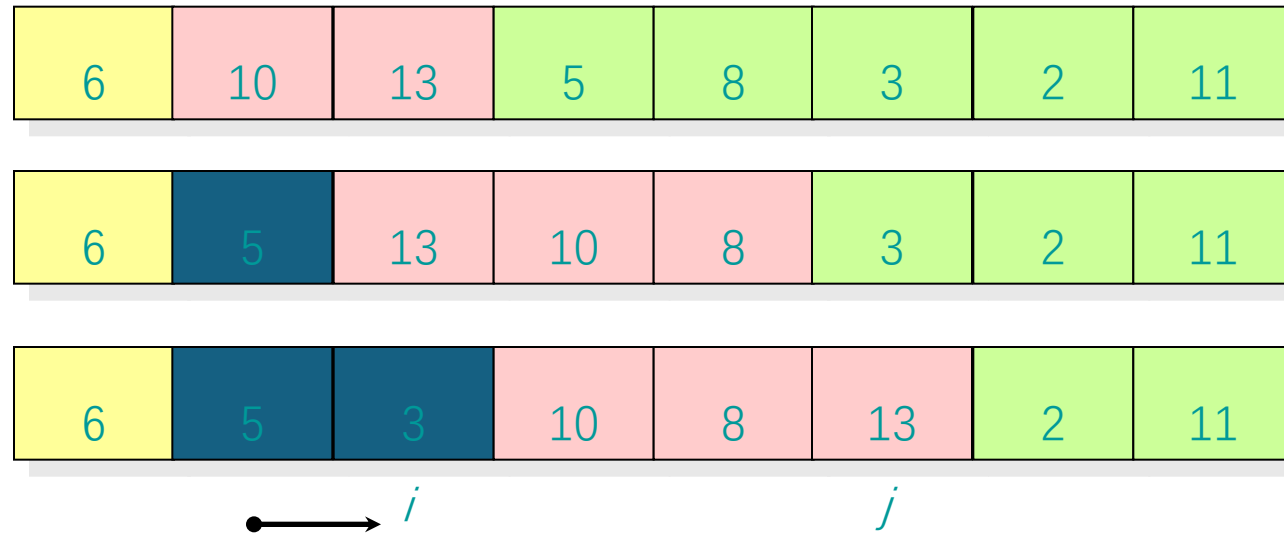
Example of partitioning



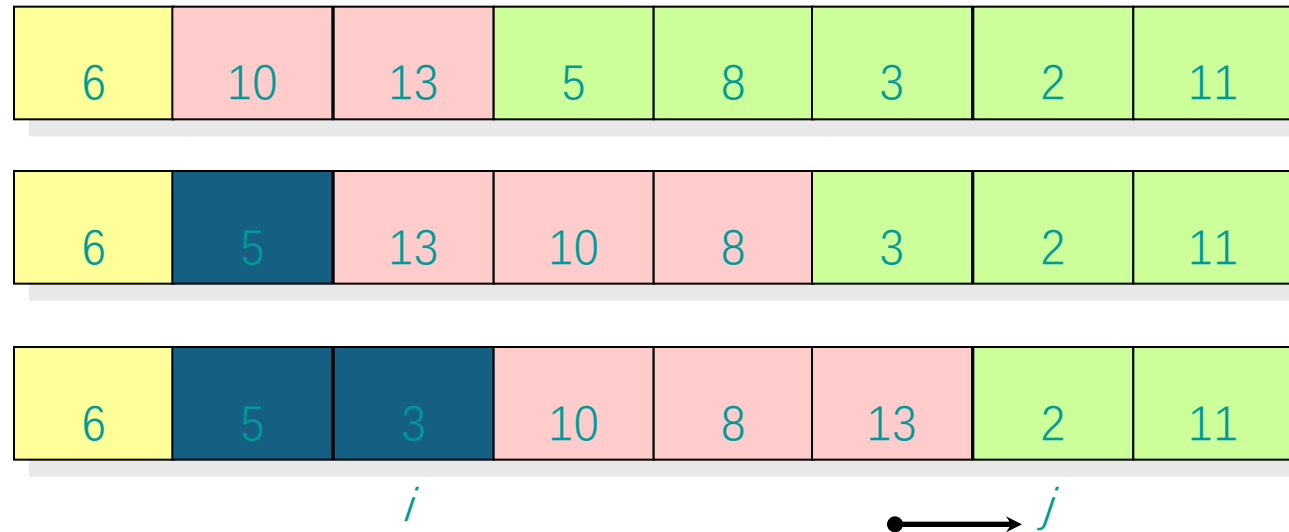
Example of partitioning



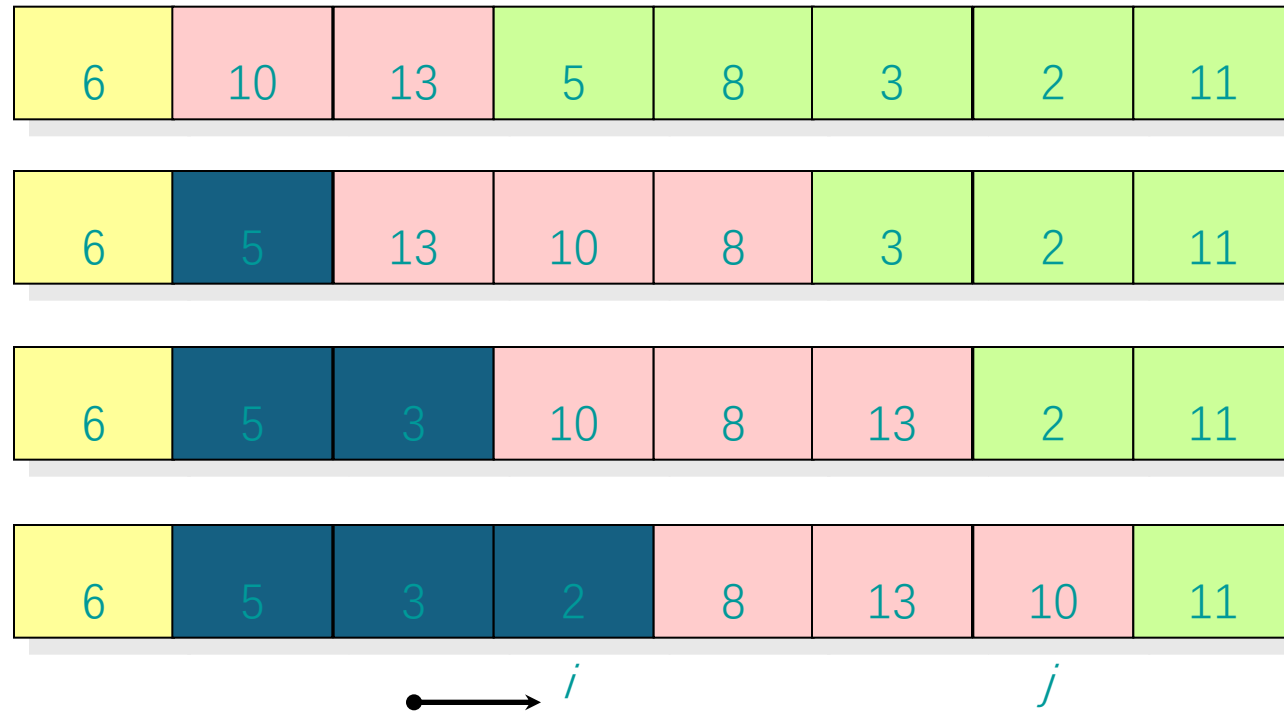
Example of partitioning



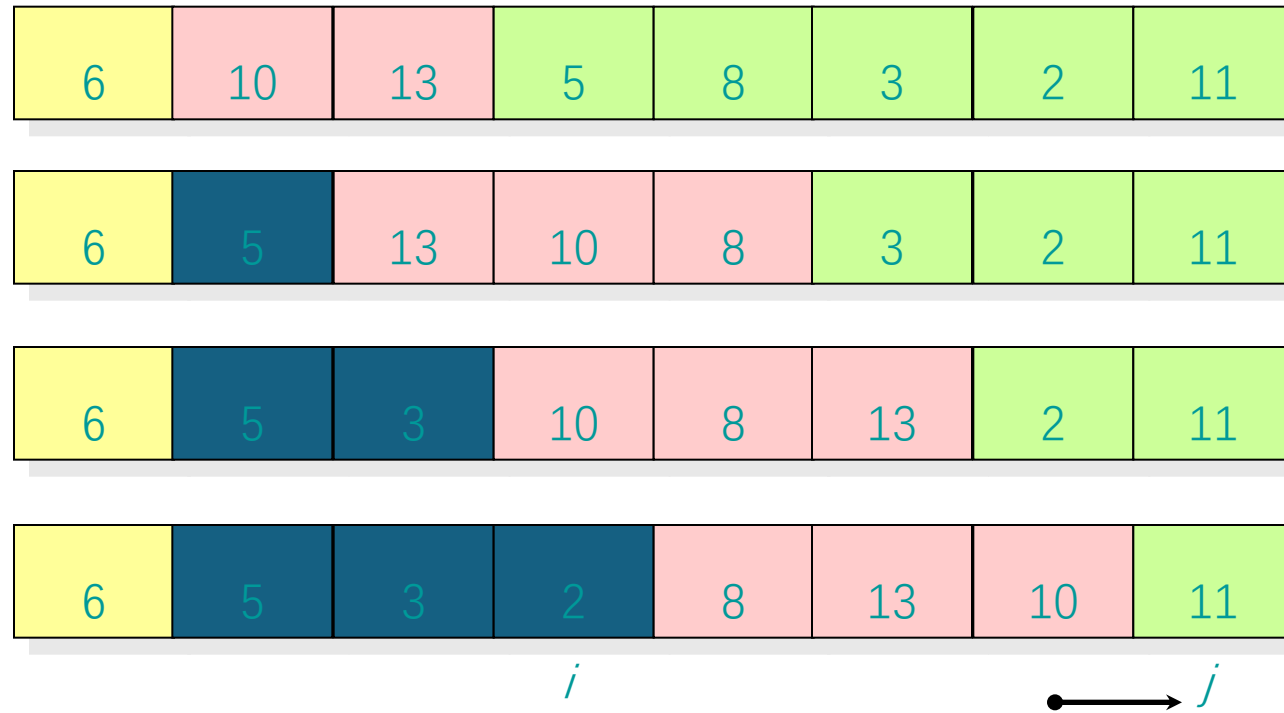
Example of partitioning



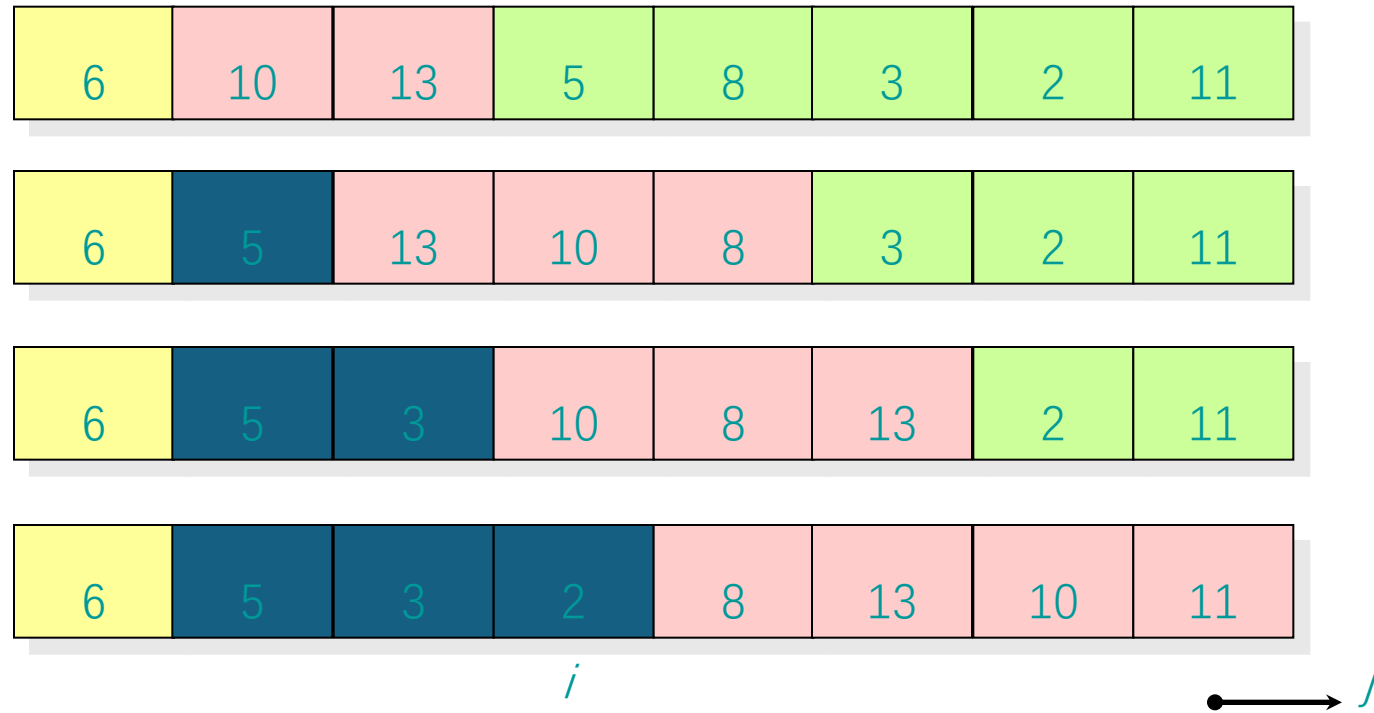
Example of partitioning



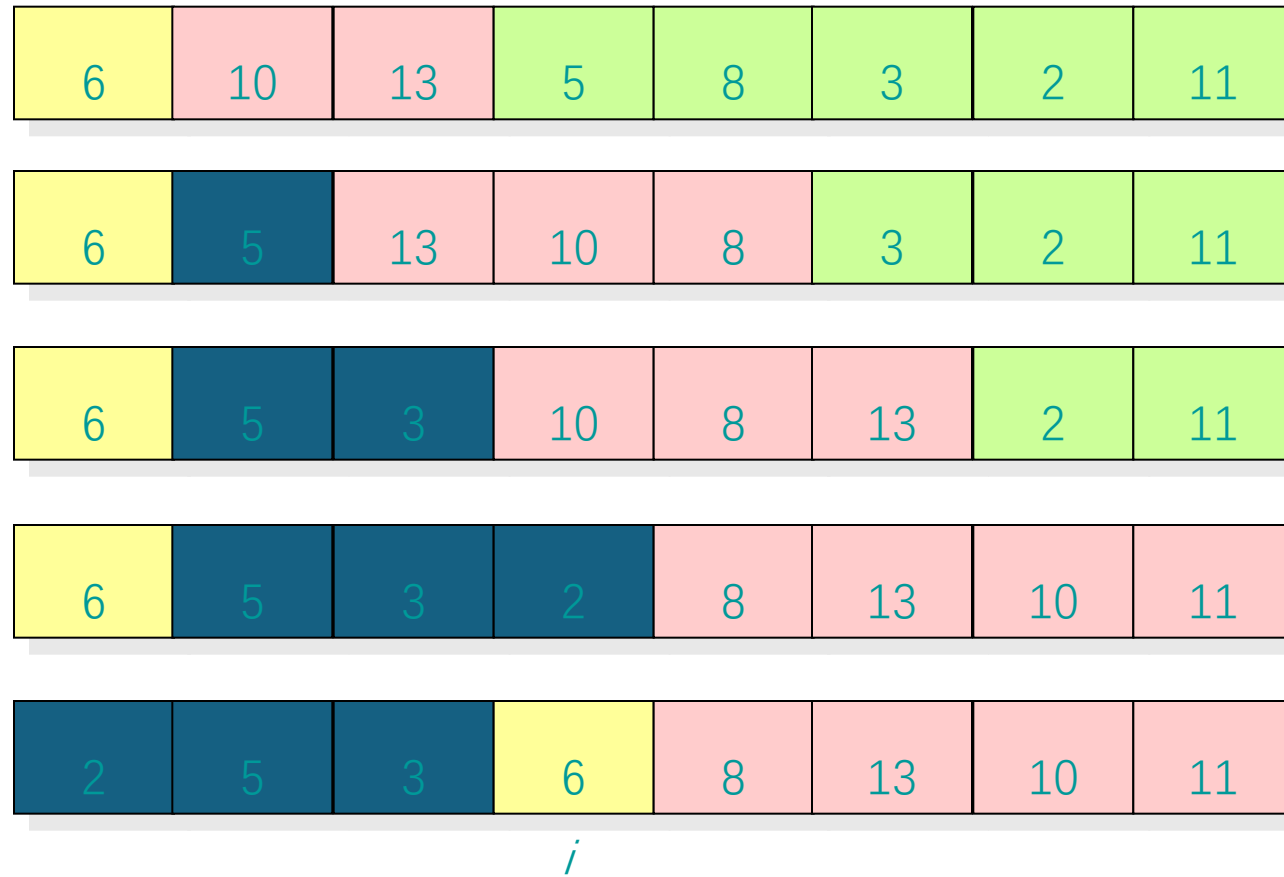
Example of partitioning



Example of partitioning



Example of partitioning



Q7. Trace Two Levels

- Array:

[4, 1, 6, 2, 5, 3]

- **Question:**

Use Quick Sort with **first element as pivot**.

Show:

- array after the **first partition**
- which subarrays are recursively processed next

Q8. Full Quick Sort Trace

- Array:

[4, 2, 5, 1, 3]

- **Question:**

Use Quick Sort with **first element as pivot**.

Show the array after **each partition step** until sorted.

Recap : The Master Theorem

Let $f(n)$ be a function that returns a positive value for every integer $n > 0$. We know:

$$f(1) = O(1)$$
$$f(n) \leq \alpha \cdot f\left(\left\lceil \frac{n}{\beta} \right\rceil\right) + \underbrace{O(n^\gamma \cdot \log^\lambda n)}_{g(n)} \quad (\text{for } n \geq 2)$$

where $\alpha \geq 1, \beta > 1, \gamma \geq 0$, and $\lambda \geq 0$ are constants. Then:

- If $\log_\beta \alpha < \gamma$, then $f(n) = O(n^\gamma \log^\lambda n)$.
- If $\log_\beta \alpha = \gamma$, then $f(n) = O(n^\gamma \log^{\lambda+1} n)$.
- If $\log_\beta \alpha > \gamma$, then $f(n) = O(n^{\log_\beta \alpha})$.

α : The number of pieces the cake is divided into each time.

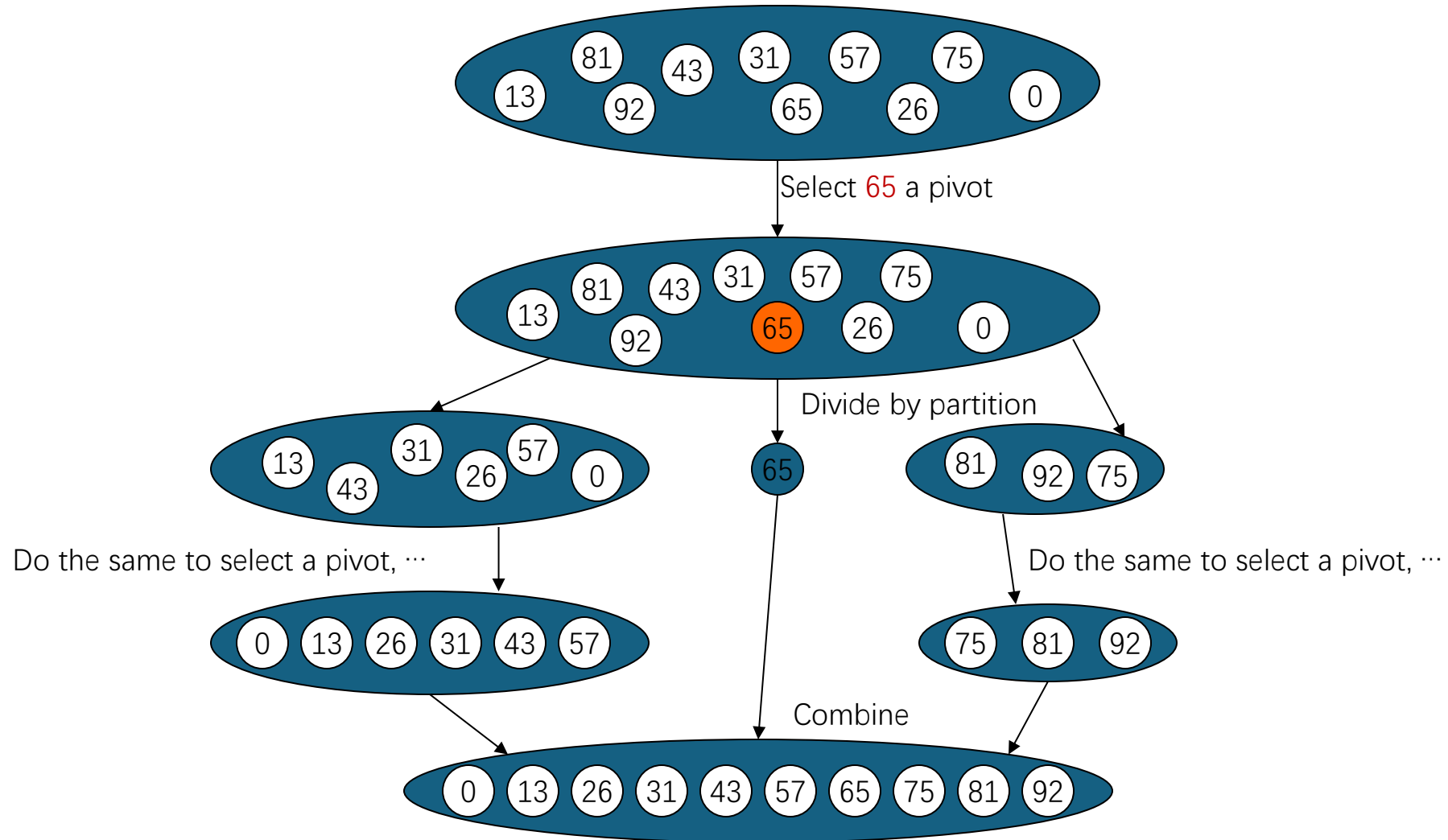
β : The factor by which the size of each piece is reduced compared to the original cake.

$g(n)$: The additional work required during each division.

The theorem can be proved by carefully applying the “expansion method” we saw earlier.

The details are tedious and omitted.

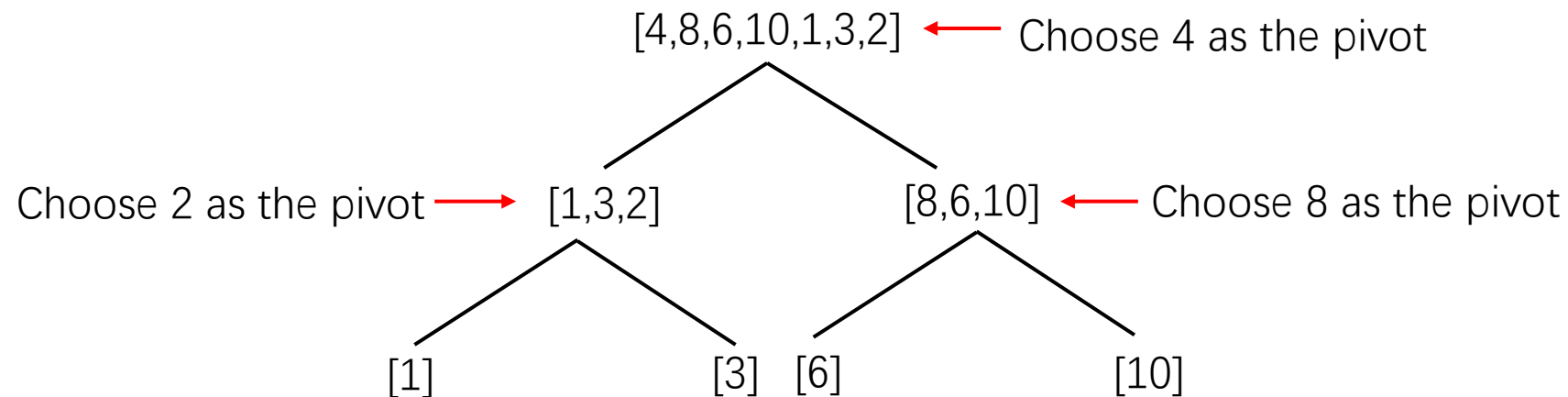
A Quick Sort Example



Best-case Analysis

A key issue is how to select the pivot.

- If the pivot is always selected as the **median**, the total time can be $O(n \log n)$ for an array of n numbers.



Q9. Best Case of Quick Sort

- **Question:**

Use Master Theorem to calculate the time complexity of quick sort at best case

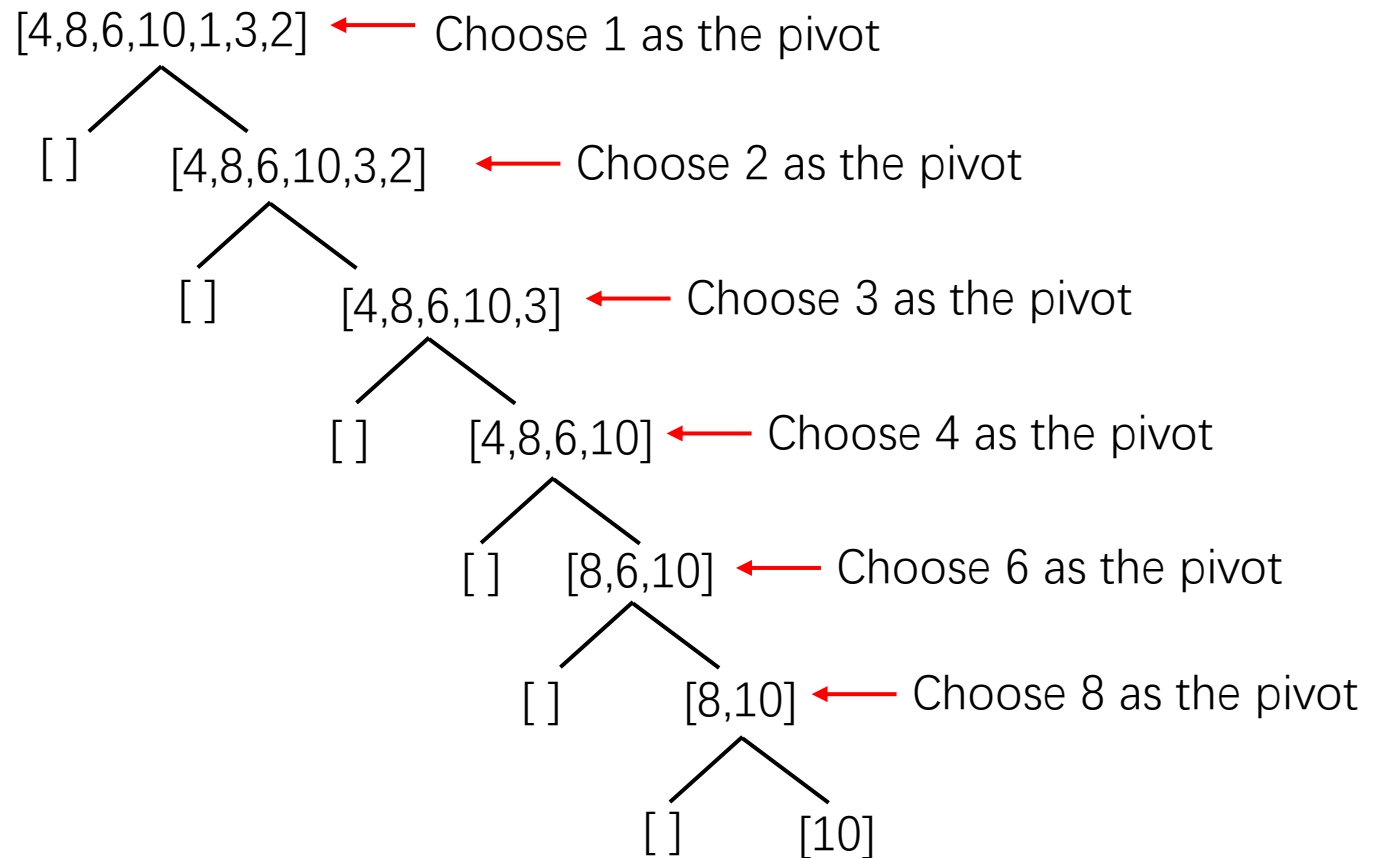
How to Select a Pivot?

- We typically use one of the following strategies:
 - Always select the **first** (or last) element as the pivot.
 - **Median of Three**: Choose the leftmost, rightmost, and middle elements in the array, and select the median among these three.
 - **Random Pivot**: Choose a random element from the array as the pivot.
- What is the performance of these strategies?

Worst-case Analysis

The worst-case running time of quick sort could be as large as $\Theta(n^2)$:

- This happens when the pivot is always selected as the **smallest or largest** number.



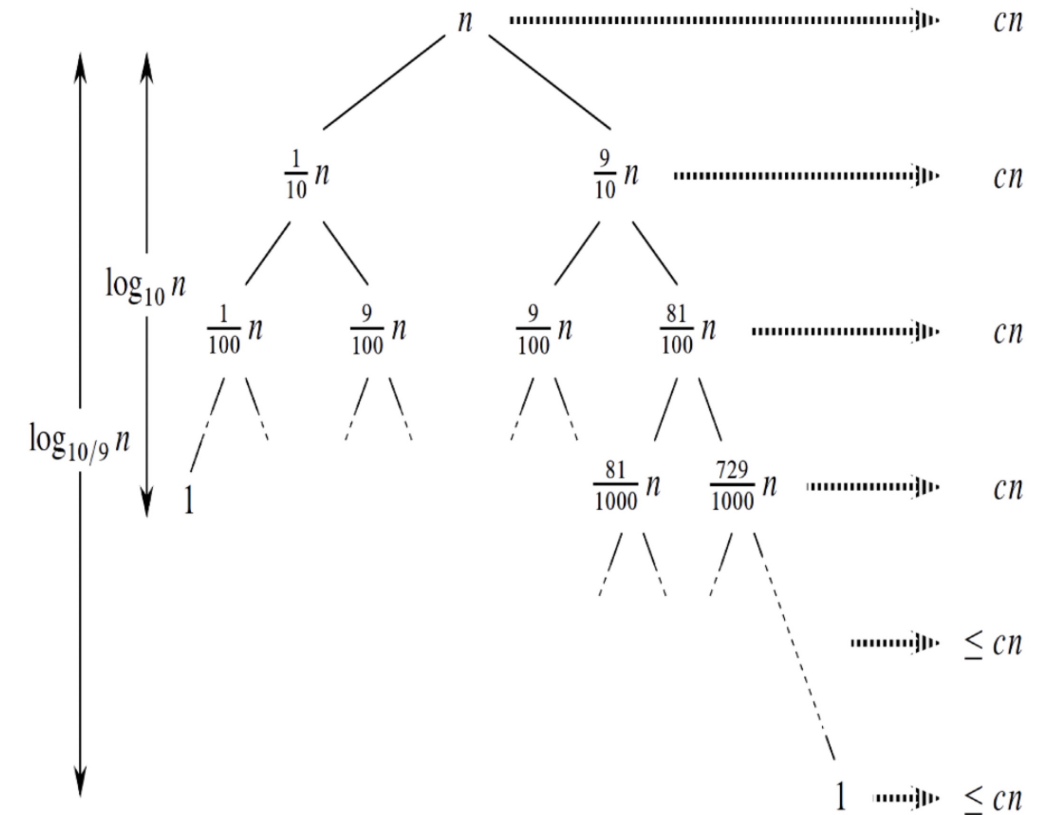
Q10. Worst Case of Quick Sort

- **Question:**

Use Master Theorem to calculate the time complexity of quick sort at worst case

Analysis of “almost-best” case

- Consider an example. Suppose that it always produces 9-to-1 split, when a pivot is selected. That is 9/10 is on one side and 1/10 is on the other side.
- Let l be deepest recursion level. We have
 - $\left(\frac{9}{10}\right)^l n \geq 1$, i.e., $l \leq \log_{\frac{10}{9}} n$.
- Thus, the time complexity is $O(n \log n)$.



$O(n \lg n)$

Quick Sort in Practice

- Randomized quick sort runs in $O(n \log n)$ time on many inputs.
- Quick sort is a practical general-purpose sorting algorithm.
- Quick sort is typically over twice as fast as merge sort.
- Quick sort can benefit substantially from **code tuning**.
- <https://www.toptal.com/developers/sorting-algorithms>

Outline

- The Master Theorem
- Quick Sort
- **Sorting Lower Bound**

Sorting Lower Bound

- We already know that n elements can be sorted in $O(n \log n)$ time.
- This lecture will prove that the time complexity is optimal for comparison-based algorithms. In other words, every such algorithm must incur $\Omega(n \log n)$ time on at least one input.

Sorting Lower Bound

- There are $n!$ different ways to permute the n elements in the input array A .
- Example: For $n = 3$, 6 permutations:

$A[1], A[2], A[3]$

$A[1], A[3], A[2]$

$A[2], A[1], A[3]$

$A[2], A[3], A[1]$

$A[3], A[1], A[2]$

$A[3], A[2], A[1]$

- The goal of sorting is essentially to decide which of $n!$ Permutations is the final sorted order.

Sorting Lower Bound: Comparison-Based Algorithm

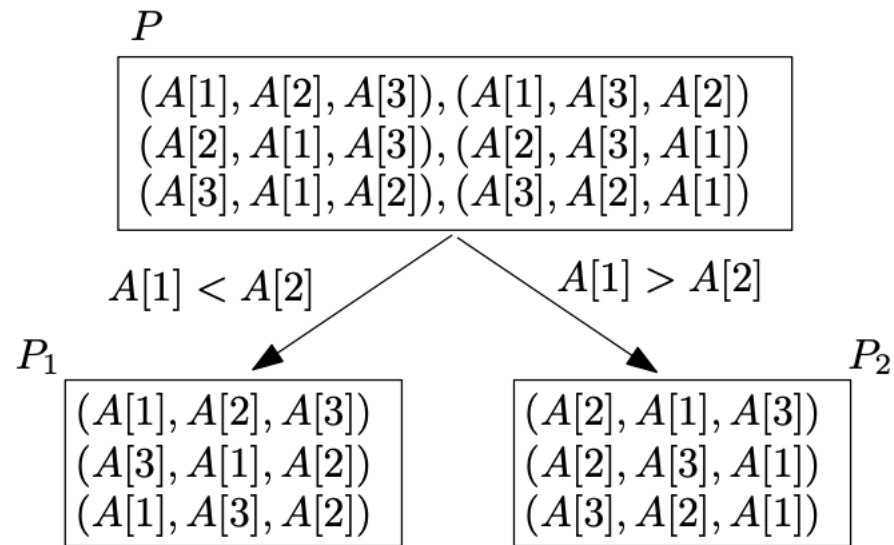
Formally, such an algorithm works by continuously shrinking a pool P of possible permutations.

- At the beginning, P contains all the $n!$ permutations.
- Every comparison allows the algorithm to discard all those permutations in P that are inconsistent with the comparison's result.
- Eventually, P has only 1 permutation left, which is thus the final sorted order.

In other words, at any moment, all the permutations that remain in P are possible results. The algorithm cannot terminate as long as $|P| \geq 2$.

Sorting Lower Bound

- Shrinking the Pool: An Example



- In general, each comparison allows us to shrink P to either P_1 or P_2 .

Sorting Lower Bound: The Framework of Comparison-Based Algorithm

Framework

1. $P \leftarrow$ all the $n!$ permutations of A
2. **while** $|P| > 1$
3. make a comparison between elements e_1 and e_2
4. **if** $e_1 < e_2$ **then**
5. $P \leftarrow P_1$, where P_1 is the set of permutations in P
 consistent with $e_1 < e_2$
6. **else**
7. $P \leftarrow P_2$, where P_2 is the set of permutations in P
 consistent with $e_1 > e_2$
8. **return** the permutation in P

Various algorithms differ in how they implement Step 3.

Sorting Lower Bound: A Worst-Case Lower Bound

Formally, such an algorithm works by continuously shrinking a pool P of possible permutations.

- Note that one of P_1 and P_2 contains at least half of the permutations in P (i.e., either $|P_1| \geq |P|/2$ or $|P_2| \geq |P|/2$).
- The worst case happens when P always shrinks to the **larger** set between P_1 and P_2 .
- In this case, the size of P shrinks by at most half after each comparison.
- Hence, the number of comparisons required before $|P|$ decreases to 1 is $\log_2(n!)$.

The next slide shows $\log_2(n!) = \Omega(n \log n)$.

Sorting Lower Bound: A Worst-Case Lower Bound

$$\begin{aligned}\log_2(n!) &= \sum_{i=1}^n \log_2 i \\ &\geq \sum_{i=n/2}^n \log_2 i \\ &\geq (n/2) \log_2(n/2) \\ &= \Omega(n \log n)\end{aligned}$$

We now conclude that any comparison-based algorithm must incur $\Omega(n \log n)$ time sorting n elements in the worst case.

RAM with Randomization

Recall that the core of the RAM model is a set of atomic operations. We can extend this set with one more atomic operation:

- **RANDOM(x, y)**: Given integers x and y (satisfying $x \leq y$), this operation returns an integer chosen **uniformly at random** in $[x, y]$.
 - Any of $x, x + 1, x + 2, \dots, y$ has the same probability of being returned

Let X be a random variable that equals the cost of an algorithm on an input. The **expected cost** of the algorithm on the input is the expectation of X .

The **worst-case expected cost** of a randomized algorithm is the maximum expected cost of the algorithm on all possible inputs of size n .

RAM with Randomization

Example:

- Problem “[Find-a-Zero](#)”: Let A be an array of n integers, among which half of them are 0. Design an algorithm to report an arbitrary position of A that contains a 0.
- For example, suppose $A = (9, 18, 0, 0, 15, 0, 33, 0)$. An algorithm can report 3, 4, 6, or 8.

A randomized algorithm:

1. **do**
 2. $r = \text{RANDOM}(1, n)$
 3. **until** $A[r] = 0$
 4. **return** r
- The algorithm finishes in $O(1)$ expected time on every input A !

Analysis of Randomized Quick Sort (Not Tested)

Input: A set S of n integers is given in an array A of length n .

Consider the **randomized** version of quicksort:

- Pick an integer p from A uniformly at random as a pivot.

We will show that the expected cost of randomized quick sort is $O(n \log n)$ on every input.

- First, convince yourself that it suffices to analyze the number X of comparisons. The running time is bounded by $O(n + X)$.
- Next, we will prove that $E(X) = O(n \log n)$.

Analysis of Randomized Quick Sort

Denote by e_i the i -th smallest integer in S . Consider e_i, e_j for any i, j such that $i \neq j$.

What is the probability that quick sort compares e_i and e_j ?

This question, which seems to be difficult at first glance, has a surprisingly simple answer. Let us observe:

- Every element will be selected as a pivot **exactly** once.
- e_i and e_j are **not** compared, if any element **between** them gets selected as a pivot **before** e_i and e_j .

For example, suppose that $i = 7$ and $j = 12$. If e_9 is the pivot, then e_i and e_j will be separated by e_9 ([think: why?](#)) and will not be compared in the rest of the algorithm.

Analysis of Randomized Quick Sort

Therefore, e_i and e_j are compared if and only if either one is the first among $e_i, e_{i+1}, \dots, e_j$ picked as a pivot.

The probability is $2/(j - i + 1)$.

Analysis of Randomized Quick Sort

Define random variable X_{ij} to be 1, if e_i and e_j are compared. Otherwise, $X_{ij} = 0$. We thus have $\Pr[X_{ij} = 1] = 2/(j - i + 1)$. That is $E[X_{ij}] = 2/(j - i + 1)$.

Clearly, $X = \sum_{i,j} X_{ij}$. Hence:

$$\begin{aligned} E[X] &= \sum_{i,j:i < j} E[X_{ij}] = \sum_{i,j:i < j} \frac{2}{j - i + 1} \\ &= 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{j - i + 1} \\ &= 2 \sum_{i=1}^{n-1} O(\log(n - i + 1)) \\ &= 2 \sum_{i=1}^{n-1} O(\log n) = O(n \log n). \end{aligned}$$

Analysis of Randomized Quick Sort

The above analysis used the following fact:

$$1 + 1/2 + 1/3 + 1/4 + \dots + 1/n = O(\log n).$$

The left-hand side is called the **harmonic series**.

Exercise 1 – Real Algorithm Interpretation

- A recursive algorithm splits a problem of size n into **4 subproblems of size $n/2$** , and the combine step costs n^2 .
 - Write the recurrence
 - Solve it using Master Theorem

Exercise 2 – Partition Result

- Using the last element as pivot, perform **one partition step** on:

[7, 2, 9, 4, 3]

- What is the array after partition, and what is the pivot's final index?

Exercise 3 — Best-Case Recurrence of Quick Sort

- Trace quick sort on: (use the first one as pivot)
- [5, 1, 4, 2, 8, 7, 6, 3]

Exercise 4 – Quick Sort Recursion Tree Shape

- Quick sort always splits an array into subarrays of sizes:

$$\frac{n}{4} \text{ and } \frac{3n}{4}$$

- and partitioning costs $\Theta(n)$.
- Write the recurrence
- Is the running time still $\Theta(n \log n)$? Explain intuitively